

Project structure

Every folder and module in the repository, what it is responsible for, and how the modules depend on one another.

Languages	Solidity · TypeScript
Frameworks	Foundry · Next.js 14 · wagmi / viem · Chainlink CRE SDK
Companion docs	Architecture · CRE settlement workflow
Updated	September 2026

1. The repository at a glance

Three independent parts — the contracts, the web app and the Chainlink workflow — share one repository, with documentation and two helper scripts alongside.

GrantLane/	
├─ contracts/	Solidity: the escrow, the CRE receiver, tests and scripts (Foundry)
│ └─ src/	Deployed contracts and the interfaces they use
│ └─ test/	Unit tests, receiver tests, fork tests against live UMA
│ └─ script/	Deploy scripts and a step-by-step demo driver
├─ apps/web/	Next.js app: pages, API routes, components, libraries
│ └─ app/	Routes – pages and API endpoints
│ └─ components/	React components
│ └─ lib/	Chain access, UMA, World ID, storage, helpers
├─ cre-workflow/	Chainlink CRE project
│ └─ settlement-workflow/	Cron workflow that settles expired claims
├─ scripts/	ABI generation and a development seeder
├─ docs/	These PDFs and supporting notes
├─ .env.example	Every configuration value, documented
├─ DEPLOYMENT.md	Deploying the web app
├─ package.json	Root commands that wrap each part
└─ README.md	

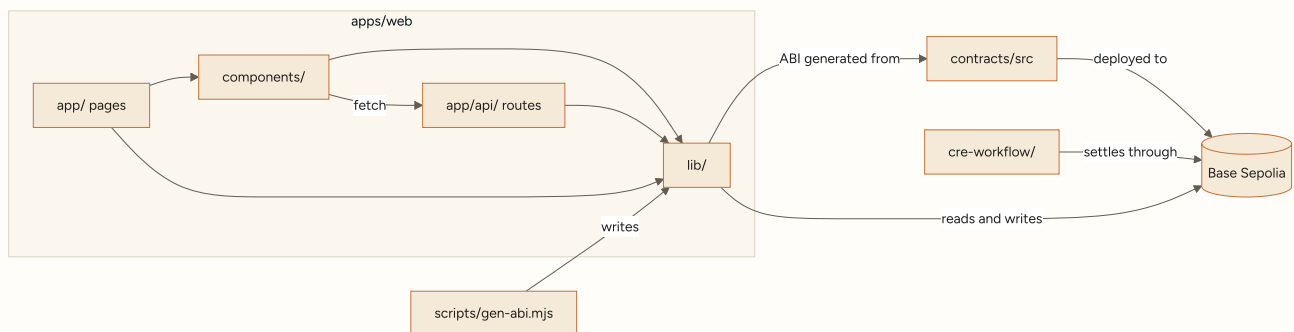


Figure 1 — How the parts depend on each other. The web app's ABI is generated from the compiled contracts.

2. contracts/

A Foundry project, Solidity 0.8.24 with OpenZeppelin 5.1. Dependencies live in `contracts/lib/` and are installed, not committed.

src/ — what is deployed

File	Responsibility
<code>GrantEscrow.sol</code>	The escrow. Funds grants, turns milestone claims into UMA assertions with a USDC bond, receives UMA's callbacks, credits approved payouts for withdrawal, exposes permissionless settlement and Automation-style upkeep, and verifies the EIP-712 Selfie Check attestations for claims and payout-wallet changes.
<code>automation/SettlementReceiver.sol</code>	Receives the CRE workflow's report and forwards it to <code>GrantEscrow.performUpkeep</code> . Can call nothing else.
<code>automation/ReceiverTemplate.sol</code>	Base for CRE receivers: accepts reports only from the configured forwarder, optionally only from an expected workflow owner or name.
<code>automation/IReceiver.sol</code>	The report-receiving interface CRE's forwarder calls.
<code>interfaces/IOptimisticOracleV3.sol</code>	The subset of UMA's oracle the escrow calls.
<code>interfaces/IOptimisticOracleV3CallbackRecipient.sol</code>	The callbacks UMA makes when a claim is disputed or resolved.
<code>interfaces/AutomationCompatibleInterface.sol</code>	<code>checkUpkeep</code> / <code>performUpkeep</code> , the upkeep interface the workflow drives.

GrantEscrow's external functions

Function	Who	What it does
<code>createGrant</code>	Grantor	Escrows the full total and commits the terms hash.
<code>submitMilestone</code>	Grantee	Checks the attestation, pulls the bond, asserts the claim on UMA.
<code>settle</code>	Anyone	Settles a claim whose window closed or whose dispute is answered.
<code>withdraw</code>	Payout wallet	Sends everything credited to the caller.
<code>assertionResolvedCallback</code>	UMA only	Approves and credits, or rejects and reopens.
<code>assertionDisputedCallback</code>	UMA only	Marks the milestone Disputed.
<code>checkUpkeep</code> / <code>performUpkeep</code>	Anyone / CRE	Finds and settles expired, undisputed claims in batches of up to 10.
<code>changePayoutWallet</code>	Grantee	Repoints payouts, gated on a Selfie Check attestation.

<code>closeGrant</code>	Grantor	Refunds unreleased funds; refused while a claim is open.
<code>setAttestor</code>	Owner	Replaces the attestor address.
<code>getGrant</code> , <code>getMilestones</code> , <code>awaitingSettlement</code>	Anyone	Views the app reads.

test/

File	What it proves
<code>GrantEscrow.t.sol</code>	45 unit tests: funding, claims, attestations, undisputed and disputed paths, callbacks, withdrawals, closing, upkeep, payout-wallet changes.
<code>mocks/MockOptimisticOracleV3.sol</code>	A mock oracle that follows UMA's bond and callback rules, for the unit tests.
<code>SettlementReceiver.t.sol</code>	8 tests: forwarder and author checks, pass-through of performData, forwarder switch.
<code>GrantEscrow.fork.t.sol</code>	7 tests against the real UMA oracle and Circle USDC on a Base Sepolia fork, including disputes and a full upkeep batch within its gas limit.

script/

File	Purpose
<code>Deploy.s.sol</code>	Deploys GrantEscrow with the oracle, USDC, attestor, dispute window and bond.
<code>DeploySettlementReceiver.s.sol</code>	Deploys the receiver pointed at an escrow and a CRE forwarder.
<code>Demo.s.sol</code>	Drives a live escrow one step per call: fund, claim, dispute, answer, settle, withdraw, status.
<code>UmaSandbox.sol</code>	Rebuilds the request data UMA's sandbox oracle needs to answer a dispute.

`foundry.toml` sets the compiler and the `base_sepolia` RPC endpoint; `remappings.txt` maps the library imports.

3. apps/web/

A Next.js 14 App Router app in TypeScript, styled with Tailwind on CSS-variable design tokens. Wallet access is wagmi and viem; storage goes through a small key-value layer.

app/ — pages

Route	Who	What it shows
/	Everyone	The landing page explaining the product.
/apply	Applicant	The application form, with Selfie Check and an autosaved draft.
/my	Applicant	Your applications and their status, with optional desktop notifications.
/applications	Grantor	The review queue.
/applications/[id]	Grantor, owner	One application: the grantor's scope-and-fund panel, or the applicant's progress.
/grants	Everyone	Every grant on the escrow and its progress.
/grant/[id]	Everyone	A grant's milestones, with claim, dispute, answer, settle and withdraw actions.
/evidence/[hash]	Everyone	The evidence behind a claim, with its hashes checked against the chain.
/admin	Grantor	The audit view: totals, open claims, disputes, deployment details.
/verify	Everyone	A standalone Selfie Check.

Also in `app/`: `layout.tsx` (fonts, theme, navigation, analytics), `providers.tsx` (wallet, data fetching and role context), `globals.css` (the design tokens) and `icon.svg`.

app/api/ — server routes

Endpoint	Purpose
GET /api/applications	Lists applications; <code>?wallet=</code> narrows to one applicant. Adds whether each funded grant is completed.
POST /api/applications	Submits an application. Requires a Selfie Check ticket; one open application per person.
GET / PATCH /api/applications/[id]	Reads one application; records approve, decline, or funded — the last only if the terms hash matches the chain.
POST /api/milestones	Prepares a claim: validates the evidence, stores it under its hash, signs the MilestoneClaim attestation.
GET /api/milestones?hash=	Returns a stored evidence bundle.
POST /api/verify-selfie	Verifies a Selfie Check proof with World and returns a ticket, or a payout-wallet attestation.

GET /api/idkit-context	Returns a request context signed with the Relying Party key, which World ID requires.
GET /api/github	Fetches a public repository snapshot for an application.

components/

Grants and milestones

GrantDetail	The grant page layout; re-reads the chain while a claim is in flight.
MilestoneCard	One milestone: claim form, dispute-window countdown, dispute, testnet answer (withheld from both sides), settle once answered.
EscrowStatus	Escrowed and released totals, open claims, bond and window.
WithdrawPanel	Shows and withdraws the connected wallet's credited payouts.
PayoutWalletPanel	Changes the payout wallet after a Selfie Check.

Applications

ApplicationForm	The application form, autosaving a draft in the browser.
ReviewPanel	Grantor: adjust milestones, approve, decline, escrow. Applicant: progress steps.
MyApplications	The applicant's list, polled for status changes.
ApplicationStageTag	One status label everywhere: In review, Scope agreed, Escrowed, Completed, Declined.
ApplicationAccess	Lets only the grantor or the applicant open an application.
GranterOnly	Hides grantor pages from other wallets.
GithubRepoCard	Repository stats snapshot.
NotifyToggle	Opt-in desktop notifications when a decision changes.

World ID

SelfieGate	"Prove you're a live human, then continue" — used before applying and claiming.
SelfieCheckRunner	Runs the IDKit request and reports its result.
SelfieCheckButton	Selfie Check for a payout-wallet change, returning an attestation.
QrCode	Renders the World App link as a scannable QR code.

Shell and presentation

NavTabs, RoleProvider, RoleSwitch	Navigation, and whether the connected wallet is a grantor.
WalletBadge, ThemeToggle	Wallet connection and network switch; light and dark themes.
Landing, Logo, Reveal	The landing page, the brand mark, scroll-in animation.
SetupNotice, ConfigDisclosure	Configuration errors, and deployment details on the audit page.

Chain and contracts

<code>chain.ts</code>	The Base Sepolia chain definition and explorer links.
<code>contracts.ts</code>	The read client, contract addresses, grant and milestone readers, status names, USDC formatting.
<code>grantEscrowAbi.ts</code>	The escrow ABI, generated from the compiled contract.
<code>useChainTx.ts</code>	Sends a transaction: switches network, waits to be mined, waits for the read node to catch up, surfaces reverts.
<code>grantProgress.ts</code>	Whether a funded grant has paid every milestone.
<code>status.ts</code>	How grant, application and milestone states are labelled and coloured.

UMA and integrity

<code>uma.ts</code>	UMA addresses and ABIs: dispute, read an assertion, check and push a dispute answer.
<code>commitments.ts</code>	Canonical terms and evidence hashing.
<code>eip712.ts</code>	The typed-data shapes the attester signs and the escrow verifies.

Identity and authorisation

<code>world.ts</code>	World ID server side: signed request contexts and proof verification.
<code>verifications.ts</code>	HMAC-signed verification tickets.
<code>attestor.ts</code>	Signs claim and payout-wallet attestations.
<code>access.ts</code>	The grantor allowlist.
<code>ticket-cache.ts</code>	Keeps a ticket through reloads so a Selfie Check isn't repeated.

Data

<code>kv.ts</code>	Key-value storage: Redis in production, a JSON file locally.
<code>applications.ts</code>	Applications: create, read, update, one-open-per-person rule.
<code>store.ts</code>	Evidence bundles, keyed by hash.
<code>github.ts</code>	Public repository lookups.
<code>form-draft.ts</code>	Browser-only form drafts.

Configuration files: `next.config.mjs`, `tailwind.config.ts`, `postcss.config.mjs`, `tsconfig.json`, `vercel.json`.

4. cre-workflow/

File	Purpose
<code>project.yaml</code>	CRE project settings: the RPCs for each target.
<code>settlement-workflow/main.ts</code>	The workflow: on each cron tick, read <code>checkUpkeep</code> and write a settlement report when claims are ready.
<code>settlement-workflow/workflow.yaml</code>	Workflow name and files per target.
<code>settlement-workflow/config.*.json</code>	Schedule, chain, escrow and receiver addresses, gas limit — for simulation (staging) and a DON deployment (production).
<code>settlement-workflow/package.json</code> , <code>tsconfig.json</code>	SDK dependency and TypeScript settings.

See the CRE settlement workflow document for how it runs.

5. scripts/ and docs/

Path	Purpose
<code>scripts/gen-abi.mjs</code>	Regenerates <code>apps/web/lib/grantEscrowAbi.ts</code> from the compiled contract.
<code>scripts/seed-applications.mjs</code>	Development only: adds sample applications. Never run it against a public deployment.
<code>docs/*.pdf</code>	This documentation.
<code>docs/base-sepolia-evidence.md</code>	Every transaction from the live runs, with explorer links.
<code>docs/demo-script.md</code>	A four-minute demo walkthrough.
<code>docs/world-feedback.md</code>	Notes from integrating World ID.
<code>docs/cre-evidence/</code>	Historical: the earlier CRE scoring design on Arc.

6. Running it

Configuration is documented in `.env.example`. Copy it and fill in the values before running anything that talks to the chain or World ID.

Web app

```
npm install --prefix apps/web
npm run dev           # http://localhost:3001
```

Contracts

```
npm run contracts:build
npm run contracts:test    # unit and receiver tests; fork tests run when an RPC is configured
```

CRE workflow

```
npm run workflow:typecheck
npm run workflow:simulate  # reads checkUpkeep, reports what it would settle
npm run workflow:broadcast # the same, sending the settlement on Base Sepolia
```

After changing the contract

```
npm run contracts:build
npm run gen:abi        # refresh the web app's ABI
```

Foundry, Bun and the CRE CLI are Linux-first; on Windows the root commands run them under WSL.